

Chapitre 11

Parallélisation, concurrence et synchronisation

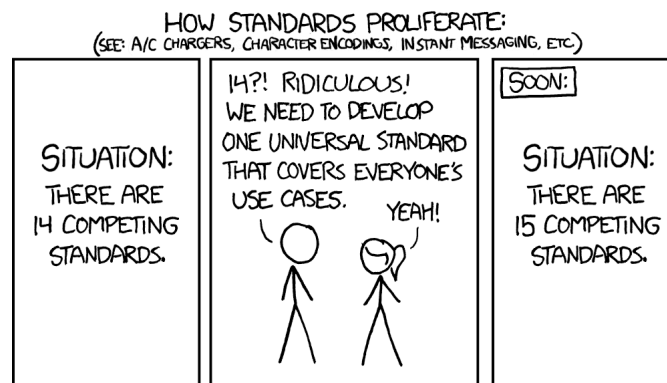


Table des matières

1	Rappels sur les processus	1
2	Fils d'exécution	1
3	Non déterminisme	3
3.1	Un premier exemple de création de thread en OCaml	3
3.2	Non déterminisme de l'exécution	3
3.3	Le même exemple en C	3
3.4	Syntaxe	4
3.5	Entrelacement	4
3.6	Somme des éléments d'un tableau	5
3.7	Produit matriciel	7
4	Les problèmes de concurrence	8
4.1	Des résultats étonnants	8
4.2	Atomicité	9
4.3	Sections critiques	9
5	Synchronisation	11
5.1	Verrous, Mutex	11
5.2	Exemple solutionné	12
5.3	Algorithme de Peterson pour implémenter un mutex avec deux threads	12
5.4	Interblocages	14
5.5	Algorithme de la boulangerie de Lamport : synchronisation d'un nombre quelconque de fils d'exécution.	14
6	Sémaphores	17
6.1	Principe et syntaxe	17
6.2	Simulation de Mutex	18
6.3	Rendez-vous	18
6.4	Producteur, Consommateur	18
6.5	Dîner des philosophes	19
6.6	Coiffeur, barbier	20
6.7	Barrière de synchronisation	21

1 Rappels sur les processus

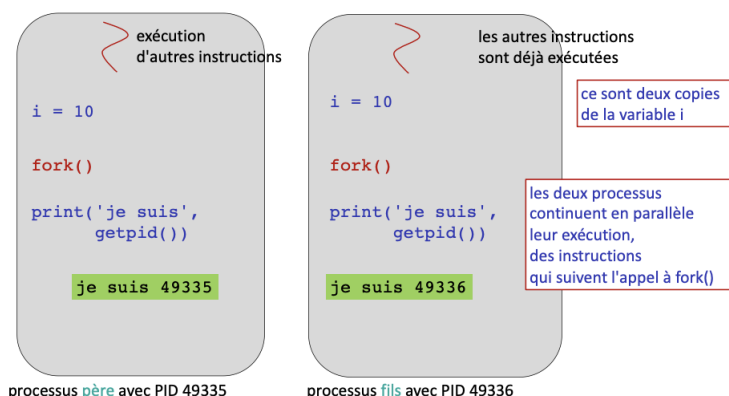
Le système d'exploitation nous fournit une vision virtuelle des ressources matérielles offertes par la machine par l'intermédiaire (entre autres) des processus, des flux d'entrée-sortie et de la mémoire virtuelle.

Un processus est un programme en cours d'exécution. C'est la ressource virtuelle que le système d'exploitation nous fournit pour se substituer au processeur (l'unité matérielle de calcul). Un processus dispose de son propre espace mémoire séparé qui lui donne l'illusion d'être le seul à utiliser cette dernière.

On rappelle que l'utilisateur écrit du code dans un langage haut niveau qui est enregistré dans un fichier et donc stocké sur le disque, ce code source est une donnée statique. Ce code est ensuite traduit en fichier exécutable qui, une fois qu'il est lancé, donne lieu à ce qu'on appelle un processus c'est-à-dire un programme en cours d'exécution. Un processus est une donnée dynamique.

L'ordonnanceur est l'outil du système d'exploitation qui permet d'organiser les accès des différents processus au processeur. Il donne accès à l'espace de calcul à un processus puis le met en attente afin de donner accès à un autre processus etc...

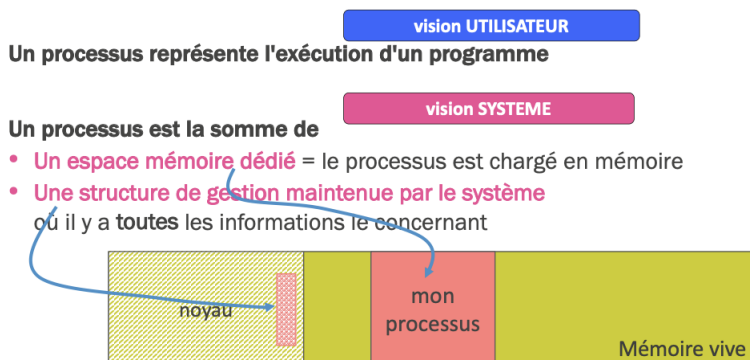
Un nouveau processus est automatiquement créé dès que l'on lance un exécutable. Une autre manière de créer un processus est d'utiliser l'appel système `fork` qui permet à un processus de se dupliquer afin de donner lieu à un nouveau processus qui en est une copie identique. Ce nouveau processus dispose de son propre espace mémoire indépendant. Le nouveau processus se retrouve exactement dans le même état que le processus père qui l'a créé, les instructions qui précèdent la commande `fork` sont donc déjà exécutées et il devra exécuter celles qui suivent l'instruction `fork`.



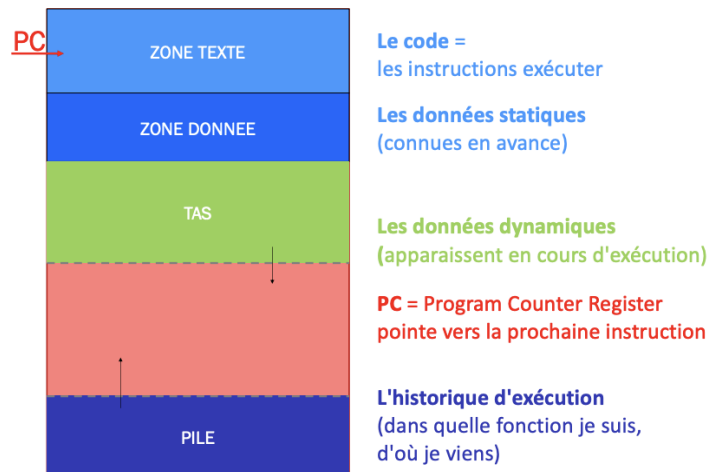
2 Fils d'exécution

Les processus dont nous venons de parler sont parfois appelés processus lourds car ils utilisent un espace mémoire dédié et sont gérés par le système d'exploitation via une structure de donnée qui contient toutes les informations le concernant. Ce type de processus nécessite donc beaucoup de ressources et ses opérations courantes (création, ordonnancement, changement d'état, terminaison) sont lentes ce qui ralentit le fonctionnement global du système.

De plus, les processus sont isolés, il est envisageable mais compliqué et coûteux de les faire "communiquer".

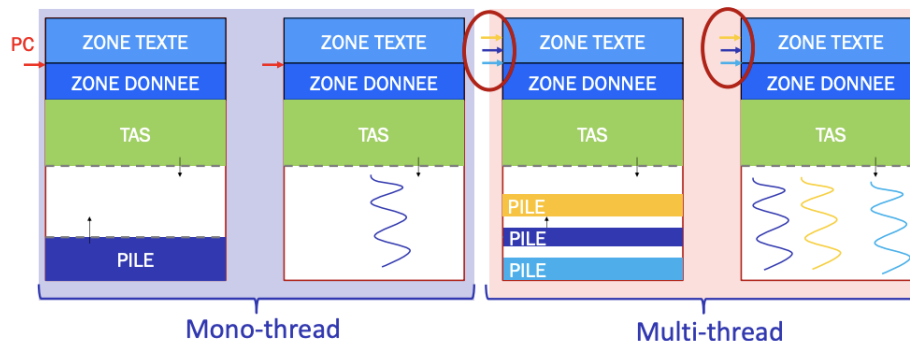


La figure suivante rappelle l'organisation mémoire d'un processus vue en première année :



Nous allons maintenant parler de **processus légers aussi appelés fils d'exécution ou threads en anglais**. Les avantages des threads sont leur efficacité et le fait qu'ils pourront se coordonner. Leur gros inconvénient est le fait qu'ils vont partager leur mémoire ce qui induit un risque majeur. En effet, si un thread modifie l'état de la mémoire au moment où un autre thread est en train de la lire alors les résultats pourront être faussés. Nous aurons donc besoin d'outils qui permettront de résoudre ces problèmes.

Lorsque l'on dispose de deux processus d'un même programme, on ne peut pas factoriser le contenu du code ni partager les données et on doit donc travailler avec deux mémoires dupliquées. Les processus sont isolés. La parallélisation de l'exécution des processus implique donc d'utiliser une grande quantité de mémoire. Les threads vont être contenus à l'intérieur d'un même processus et vont permettre d'être exécutés indépendamment comme le seraient des processus distincts tout en partageant de l'information qui sera alors sauvegardée dans le processus qui les contient. Au sein d'un même processus, les différents threads partagent le même code et la même mémoire dynamique (le tas).



On retiendra que :

1. Les threads sont des entités d'exécution plus légères que les processus
2. Ils évoluent au sein des processus
3. Ils partagent les ressources fournies par les processus
4. La programmation concurrente avec threads est difficile!

3 Non déterminisme

Nous allons ici introduire la syntaxe relative aux tread et regarder de premiers exemples.

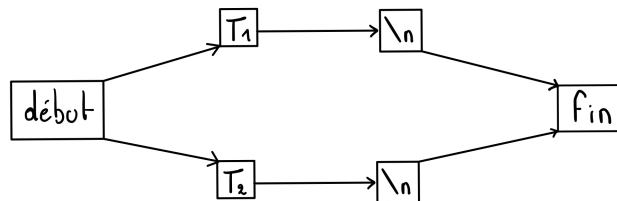
En C, on utilisera la librairie `pthread.h`.

En OCaml, on utilisera le type `Thread.t` de la librairie `Thread`.

3.1 Un premier exemple de création de thread en OCaml

```
1 let f (n : string) =
2   print_string(n);
3   print_string("\n");;
4
5 print_string("debut");;
6 let t1 = Thread.create f "thread 1";;
7 let t2 = Thread.create f "thread 2";;
8 Thread.join t1;;
9 Thread.join t2;;
10 print_string("fin");;
```

Les deux fonctions sont les seules à retenir et sont **fondamentales**. La fonction *join* sert à attendre que le thread soit fini.



Ainsi, une exécution possible correspond à un tri topologique du graphe.

3.2 Non déterminisme de l'exécution

En exécutant le programme précédent à plusieurs reprises, on peut observer que l'ordre des appels n'est pas toujours le même. En effet, **une fois les thread créés, l'ordonnancement de leurs exécutions respectives est non déterministe**.

3.3 Le même exemple en C

```
1 #include <stdio.h>
2 #include <assert.h>
3 #include <pthread.h>
4
5 void* mythread (void* arg){
6   printf("%s\n", (char*)arg);
7   return NULL;
8 }
9
10 int main(void* arg){
11   pthread_t p1, p2;
12   printf("main begins\n");
13
14   pthread_create(&p1, NULL, mythread, "thread 1" );
15   pthread_create(&p2, NULL, mythread, "thread 2");
16
17   pthread_join(p1,NULL);
18   pthread_join(p2,NULL);
19   printf("main ends \n");
20   return 0;
21 }
```

3.4 Syntaxe

Dans chacun des deux langages, on définit la fonction sur laquelle on lancera les threads puis on crée le thread et enfin on utilise une fonction `join` qui suspend l'exécution du thread appelant (votre programme principal) jusqu'à ce que le thread en argument soit terminé.

En OCaml, on utilise le type `Thread.t` et en C le type `pthread_t`.

En OCAML, la fonction `Thread.create` prend en entrée la fonction accompagnée de son entrée et la fonction `Thread.join` prend en entrée le thread. En C, le passage d'une fonction en argument étant quelque chose de subtil les fonctions `pthread_create` et `pthread_join` ont des prototypes un peu compliqués :

```
pthread_create(pthread_t* thread, pthread_attr_t* attr, void* (*f)(void*), void* arg)
et
pthread_join(pthread_t thread, void** ret)
```

Pour l'utilisation de `create` : le premier argument est l'adresse d'un thread préalablement déclaré, le deuxième argument correspond à des options auxquelles nous ne nous intéresserons pas et donc cet argument vaudra systématiquement `NULL` pour nous, le troisième argument est la fonction qui doit être exécutée par le thread et le dernier argument est l'argument avec lequel appeler cette fonction.

Avant de créer un thread, on écrira donc une fonction souvent appelée `mythread` de prototype :

```
(void*) mythread (void* in)
```

On remarquera que l'entrée et la sortie étant de type `void*` on fera des cas avec les entrées et sorties réelles. On remarquera de plus que la fonction ne peut prendre qu'un seul argument en entrée, si on veut simuler des fonctions avec plusieurs arguments il faudra créer une structure contenant tous les arguments souhaités et la fonction `mythread` prendra en entrée un pointeur sur une telle structure.

La fonction `join` prend en entrée le thread et une adresse à laquelle récupérer la valeur calculée par la fonction. Le programme stipule qu'on ne doit pas se servir de cela et limiter l'utilisation à un second paramètre qui vaut `NULL`. Ceci implique de travailler avec des variables globales à modifier si on veut récupérer les résultats des threads.

Un dernier point important, **pour utiliser les thread on doit utiliser des options de compilation** :

— en OCaml, on pourra compiler avec la commande suivante :

```
ocamlopt -I +threads unix.cmxa threads.cmxa fichier.ml -o executable
```

— en C, on ajoutera le flag `-pthread` dans les options

3.5 Entrelacement

Une fois les threads créés, l'ordonnancement de leurs actions est non déterministe : l'un des thread prend la main puis se met en pause pour que le second travaille et ainsi de suite avec des changements qui obéissent aux règles édictées par l'ordonnanceur qui sont non déterministes.

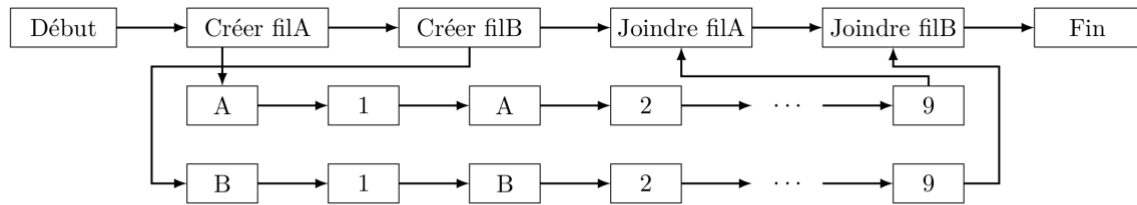
Définition - Algorithme séquentiel ou concurrent

Un algorithme ou un programme est dit **séquentiel** si les instructions qui le composent sont exécutées l'une après l'autre, toujours dans le même ordre. Il est dit **concurrent** s'il contient plusieurs unités, qu'on appelle fil d'exécution ou thread, chaque fil s'exécutant séquentiellement.

Remarque : Dans un programme concurrent, il n'y a donc pas la garantie que les instructions seront exécutées toujours dans le même ordre.

Reprenons le premier exemple et étoffons le un tout petit peu pour mettre cela en évidence :

```
1 let limite = 9;;
2 let f n =
3   for i= 1 to limite do
4     print_string(n); print_int(i); print_string("\n");
5   done;;
6
7 print_string("debut");;
8 let t1 = Thread.create f "A";;
9 let t2 = Thread.create f "B";;
10 Thread.join t1;;
11 Thread.join t2;;
12 print_string("fin");;
```



La seule contrainte sur l'exécution du programme est d'exécuter les instructions dans un ordre topologique du graphe. Cependant, comme il existe plusieurs ordres topologiques, plusieurs exécutions sont possibles. On peut imaginer que tous les affichages suivants sont possibles.

A1A2A3A4A5A6A7A8A9B1B2B3B4B5B6B7B8B9
 AB11BA2A23B3BA44BA55B6AB76AB7A88B9A9
 B1BA2B13BA24A3A4A5AB5B6A7A68B7A9B8B9

De même, il est tout à fait possible d'imaginer que le fil A commence ses instructions après avoir fini toutes les instructions du fil B.

Ainsi, **l'exécution d'un programme concurrent est non-déterministe** : il n'y a pas qu'une seule exécution possible.

3.6 Somme des éléments d'un tableau

Écrire une fonction qui calcule la somme des éléments d'un tableau passé en entrée.

```

1 let f (tab : int array) : int =
2   let res = ref 0 in
3   for i = 0 to Array.length tab - 1 do
4     res := !res + tab.(i)
5   done;
6   !res

```

Écrire une fonction qui calcule ce résultat en utilisant 2 thread.

En OCaml

```

1 let f (debut,fin,tab,res) =
2   for i = debut to fin do
3     res := !res + tab.(i)
4   done
5
6
7 let main () =
8   print_string "debut\n";
9   let main_tab = Array.make 10 1 in
10  let res1 = ref 0 in
11  let res2 = ref 0 in
12  let t1 = Thread.create f ( 0, 4, main_tab, res1) in
13  let t2 = Thread.create f ( 5, 9, main_tab, res2) in
14  Thread.join t1;
15  Thread.join t2;
16  print_int (!res1 + !res2);
17  print_newline ();
18  print_string "fin\n"
19
20 let _ = main ()

```

En C

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include <stdlib.h>
5
6  struct data_s{
7      int debut;
8      int fin;
9      int* tab;
10     int* tmp_res;
11 };
12 typedef struct data_s data;
13
14 void* mythread(void* arg){
15     data* arg_tmp = (data*)arg;
16     for (int i=arg_tmp->debut; i<arg_tmp->fin +1; i++){
17         *(arg_tmp->tmp_res) = *(arg_tmp->tmp_res) + (arg_tmp->tab)[i];
18     }
19     return NULL;
20 }
21
22 int main(){
23     // Initialisation des variables
24     pthread_t p1,p2;
25     int* tab = malloc(10 * sizeof(int));
26     for (int i=0 ; i<10; i++){
27         tab[i]=i;
28     }
29     int tmp1 = 0;
30     int tmp2 = 0;
31
32     data* data1 = malloc(sizeof(data));
33     data* data2 = malloc(sizeof(data));
34     data1->debut = 0;
35     data1->fin = 4;
36     data1->tab = tab;
37     data1->tmp_res = &tmp1;
38
39     data2->debut = 5;
40     data2->fin = 9;
41     data2->tab = tab;
42     data2->tmp_res = &tmp2;
43
44     // Debut du main
45     printf("main begins \n");
46
47     pthread_create(&p1,NULL,mythread,data1);
48     pthread_create(&p2,NULL,mythread,data2);
49
50     pthread_join(p1,NULL);
51     pthread_join(p2,NULL);
52
53     int res = tmp1 + tmp2;
54     printf("Somme : %d \n", res);
55     printf("main ends \n");
56     return 0;
57 }
58 }
```

Écrire une fonction qui le fait avec n threads.

```
1 let f (tab, debut, fin, tab_res, index_tab_res) =
2   for i = debut to fin do
3     tab_res.(index_tab_res) <- tab_res.(index_tab_res) + tab.(i)
4   done
5
6 let somme_tab (tab : int array) : int =
7   let res = ref 0 in
8   for i = 0 to Array.length tab -1 do
9     res := !res + tab.(i)
10  done;
11  !res
12
13 let main () =
14   print_string "debut\n";
15   let main_tab = Array.make 10 1 in
16   let n = 3 in
17   let len_main_tab = Array.length main_tab in
18   let sub_len = len_main_tab // n in
19   let tab_res = Array.make n 0 in
20   let thread_tab = Array.init n (fun i-> Thread.create f (main_tab,i*sub_len, (i+1*sub_len)-1, tab_res, i))
21   in
22   Array.iter (fun t-> Thread.join t) thread_tab;
23   somme_tab res_tab
```

3.7 Produit matriciel

```
1 let cij (a,b,i,j,c) =
2   let r = Array.length a.(0) in
3   for k=0 to r-1 do
4     c.(i).(j) <- c.(i).(j) + a.(i).(k) * b.(k).j ;
5   done
6
7 let main () =
8   let n = 10 in
9   let m = 10 in
10  let a = Array.make_matrix n n 1 in
11  let b = Array.make_matrix n n 1 in
12  let c = Array.make_matrix n m 0 in
13  let tab_thread = Array.init n (fun i-> Array.init m (Array.init m (fun j -> Thread.create cij (a,b,i,j,c))
14  )) in
15  Array.iter (fun t -> Array.iter Thread.join t) tab_thread ;
16  c
```

4 Les problèmes de concurrence

Nous allons maintenant mettre en évidence les problèmes de concurrence qu'induisent l'utilisation des threads.

4.1 Des résultats étonnants

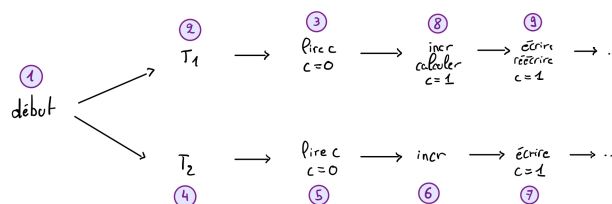
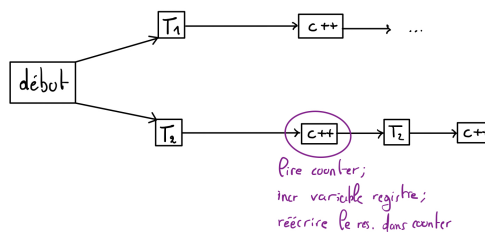
Exécutons le code suivant :

Un exemple.

```
1 #include <stdio.h>
2 #include <assert.h>
3 #include <pthread.h>
4
5 static int counter = 0;
6
7 void* mythread (void* arg){
8     printf("%s : begin\n", (char*)arg);
9     for (int i=0; i<9999; i++){
10         counter++;
11     }
12     printf("%s : done\n", (char*)arg);
13     return NULL;
14 }
15
16 int main(void* arg){
17     pthread_t p1, p2;
18     printf("main begins, counter = %d\n", counter);
19
20     pthread_create(&p1, NULL, mythread, "thread 1");
21     pthread_create(&p2, NULL, mythread, "thread 2");
22
23     pthread_join(p1, NULL);
24     pthread_join(p2, NULL);
25
26     printf("main ends, counter = %d \n", counter);
27     return 0;
28 }
```

On remarque alors que la somme renvoyée n'est jamais la même et n'est donc pas 20000! Mais alors,

Comment expliquer ce phénomène?



Voici le code assembleur de `counter++` :

```
1 mov counter, %eax
2 addl $1, %eax
3 movl %eax, counter
```

Ainsi, suivant l'entrelacement obtenu lors de l'exécution, les résultats attendus ne sont pas forcément corrects.

Exercice 1 Déterminer les valeurs minimales et maximales que peut prendre n à la fin de l'exécution des deux threads.

— **Valeur maximale** : 20000

si le Thread 1 exécute toute la fonction puis le thread 2 l'exécute aussi intégralement et on ne peut pas faire mieux car c'est le nombre total d'incrémentations

— **Valeur minimale** : 2

`i` est locale c'est à dire propre à chaque thread. `counter` est globale donc partagée.

T_1	T_2
rentre dans la boucle <code>i = 0</code> lire <code>counter = 0</code>	
pose <code>counter = 1</code>	effectue tout sauf un tour de boucle de la fonction → <code>counter = 9999</code>
va au bout de la fonction <code>counter = 10 000</code>	entre dans son dernier tour de boucle lit <code>counter = 1</code>
	termine son dernier tour de boucle ... pose <code>counter = 2</code>

4.2 Atomicité

Définition - Opération atomique

Une **opération atomique** est une opération qui ne peut pas être interrompue. Un thread en train d'effectuer cette opération ne peut pas passer la main tant qu'elle n'est pas terminée.

Remarque : Seules les instructions machines sont atomiques.

→ Nous allons tout de même voir par la suite comment rendre des instructions atomiques.

4.3 Sections critiques

Une section critique est une portion de code qui, pour garantir la sûreté du résultat, ne peut être exécutée simultanément que par un nombre maximal de threads différents (en pratique nous nous restreindrons à un seul thread).

Par exemple, l'instruction `counter++` du code précédent est une section critique.

Plus la section critique est large, plus la sûreté est garantie, mais une grande section critique empêche les autres threads de s'exécuter et donc ralentit l'exécution globale, il est donc nécessaire d'avoir les sections critiques les plus petites possibles (contenant le moins d'instructions possibles) mais garantissant l'absence de concurrence.

Exercice 2 On considère une variable n initialisée à 0 et trois fils d'exécutions qui effectuent les opérations suivantes.

- $T_1 : a \leftarrow n, n \leftarrow 1, b \leftarrow n$
- $T_2 : c \leftarrow n, n \leftarrow 2, d \leftarrow n$
- $T_3 : e \leftarrow n, f \leftarrow n$

On suppose que l'instruction $x \leftarrow y$ consiste à écrire le contenu de y dans x et est une instruction atomique. Après l'exécution de ces trois fils :

1. que peut valoir c ?

$c = 0, 1$

2. que peut valoir b ?

$b = 1, 2$

3. que peut valoir e ?

$e = 0, 1, 2$

4. si c vaut 1, que peut valoir d ?

$d = 2$

5. si f vaut 0, que peut valoir e ?

$e = 0$

6. si a vaut 2 et d vaut 1, que peut valoir b ?

$b = 1$

Exercice 3 On considère le code suivant :

```

1  int k = 0, i = 0;
2
3  void* foo(void* arg){
4      while (i < 10){
5          i++;
6          k++;
7      }
8      return NULL;
9  }
10
11 int main(void){
12     pthread_t t1, t2;
13     pthread_create(&t1, NULL, foo, NULL);
14     pthread_create(&t2, NULL, foo, NULL);
15     pthread_join(t1, NULL);
16     pthread_join(t2, NULL);
17     return 0;
18 }
```

Quelles sont les valeurs minimale et maximale que peut prendre théoriquement k à la fin de l'exécution?

MIN	
T_1	T_2
lire $i = 0$ poser $i = 1$ lire $k = 0$	effectue la fonction intégralement $i = 10, k=9$
pose $k = 1$ lit $i = 10$ stoppe la boucle while	

T_1	MAX	T_2
lit $i = 0$ entre dans la boucle fixe $i = 0$ avant incrémentation		exécuter toute la fonction jusqu'à obtenir $k = i = 10$ s'arrête avant d'avoir vérifié qu'il avait fini (que la condition du while est fausse)
pose $i = 1$ (fin de $i++$) pose $k = 11$ ($k++$)		lit $i = 1$ décide de rentrer dans la boucle, dans $i++$ il a lu $i = 1$
Effectuer le programme sauf la dernière évaluation de la condition $i = 10$ et $k = 20$		pose $i = 2$ et $k = 21$
lit $i = 2$, rentre dans la boucle lit $i=2$ (dans la partie $i++$)		effectue 8 tours $i = 10$ et $k = 29$
pose $i = 3$ pose $k=30$		lit $i=3 \dots$

Finalement :

$$10 + 9 + 1 + 8 + 1 + \dots + 1 = 65$$

5 Synchronisation

5.1 Verrous, Mutex

Pour réaliser une *section critique*, on utilise une primitive appelée *verrou* ou **mutex**. Un verrou est une variable qui dispose de deux opérations : **lock** et **unlock**. Un seul thread peut disposer du verrou. Pour délimiter une section critique, on utilise le principe suivant :

```

1 lock(m)
2 <section critique>
3 unlock(m)

```

Ainsi, lorsqu'un thread entre dans la section critique, le verrou est bloqué et aucun autre thread ne peut rentrer dans la même section de code avant qu'il ne l'est déverrouillé.

Il existe plusieurs manières d'implémenter en pratique les verrous mais chacune d'entre elle doit garantir les points suivants :

1. un seul thread peut être dans la section critique (**exclusion mutuelle**)
2. un thread ne peut pas attendre indéfiniment pour entrer en section critique (**absence de famine**)
3. un thread en dehors de la section critique ne peut pas bloquer les autres threads (**pas d'interblocage**)

Voici la syntaxe que l'on pourra utiliser :

1. En OCaml, on utilise le type **Mutex.t**
 - (a) **Mutex.create** : **unit** -> **Mutex.t** crée un nouveau mutex.
 - (b) **Mutex.lock** : **Mutex.t**->**unit** verrouille le mutex passé en argument.
 - (c) **Mutex.unlock** : **Mutex.t**->**unit** déverrouille le mutex passé en argument.
2. En C, on utilise le type **pthread_mutex_t**
 - (a) pour initialiser un mutex lors de sa déclaration, on utilisera : **pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER**
 - (b) **pthread_mutex_lock** prend en argument l'adresse d'un mutex et le verrouille.
 - (c) **pthread_mutex_unlock** prend en argument l'adresse d'un mutex et le déverrouille.

L'idée fondamentale est qu'**un seul thread à la fois peut verrouiller le mutex**, ainsi si un thread essaie de verrouiller un verrou déjà verrouillé alors il est mis en attente. La fonction `unlock` réveille les threads éventuellement mis en attente qui seront alors en compétition pour acquérir le verrou.

5.2 Exemple solutionné

On suppose avoir déclaré la variable globale `pthread_mutex_t v`. En reprenant l'algorithme 1.

```
1 void* mythread (void* arg){
2     printf("%s : begin\n", (char*)arg);
3     for (int i=0;i<9999;i++){
4         pthread_mutex_lock(&v);
5         counter++;
6         pthread_mutex_unlock(&v);
7     }
8     printf("%s : done\n", (char*)arg);
9     return NULL;
10 }
```

5.3 Algorithme de Peterson pour implémenter un mutex avec deux threads

La réalisation pratique d'un mutex est compliquée. Il existe néanmoins plusieurs solutions qui pour la plupart se basent sur des ressources matérielles.

Nous allons ici voir une solution algorithmique et donc purement logicielle qui nous permettra de comprendre les enjeux et les difficultés liés à la conception d'une telle primitive. Cet algorithme, inventé par Peterson en 1981, ne fonctionne que pour deux threads et recourt à l'attente active : lorsqu'un thread est bloqué avant d'accéder à la section critique, il doit continuer de travailler au sein d'une boucle plutôt que d'être mis en veille. Cette attente active est peu efficace.

Attention, on étudie ici des solutions d'un point de vue théorique, mais il est généralement fortement déconseillé de réimplémenter ces solutions soi-même : les mutex existent déjà en OCaml et en C, donc autant les utiliser.

Solution 1 L'idée principale est d'utiliser un tableau de deux booléens indiquant pour chaque thread s'il est ou non en section critique :

```
1 bool flag[2]={false,false};
2 void lock(int i){
3     int other = 1-i;
4     while (flag[other]) ; //attente active
5     flag[i]=true;
6 }
7
8 void unlock(int i){
9     flag[i]=false;
10 }
```

Quel est le problème rencontré avec cette solution?

T_0 lock	T_1 lock
<code>other0 = 1</code>	<code>other1 = 0</code>
<code>lit flag[1] = false</code>	<code>lit flag[0] = false</code>
<code>pose flag[0] = true</code>	<code>pose flag[1] = true</code>
...	...
Pas d'exclusion mutuelle	

Solution 2 On modifie donc la sémantique du tableau qui contiendra maintenant `true` si les threads veulent ou non entrer en section critique :

```
1 bool want[2]={false,false};
2
3 void lock(int i){
4     int other = 1-i;
```

```

5     want[i]=true;
6     while (want[other]) ; //attente active
7 }
8
9 void unlock(int i){
10    want[i]=false;
11 }

```

Interblocage : si les deux threads posent `want` à `true` ils resteront bloqués dans le `while` indéfiniment tous les deux. Pourquoi a-t-on effectivement exclusion mutuelle?

1. T_0 pose `want[0] = true`
2. T_1 pose `want[1] = true`
3. T_0 lit `want[1] = false`
4. T_1 lit `want[0] = false`

On sait que (1) < (3) et (2) < (4).

Supposons par l'absurde qu'il n'y ait pas d'exclusion mutuelle. Alors (3) < (2) et (4) < (1). On obtient alors une exécution de la forme :

$$(1) < (3) < (2) < (4) < (1)$$

Ce qui est absurde car cela ne peut correspondre à une exécution valide.

Solution 3 On va alors proposer l'utilisation d'une variable `turn` qui permet de déterminer lequel des deux threads peut acquérir le verrou :

```

1 int turn = 0;
2
3 void lock(int i){
4     int other = 1-i;
5     while (turn == other) ; //attente active
6 }
7
8 void unlock(int i){
9     turn = 1-i;
10 }

```

Ici, un problème apparaît encore : l'alternance risque de créer un blocage si un thread a terminé et l'autre a besoin de rentrer plusieurs fois en section critique.

Solution correcte La solution de Peterson combine l'utilisation du tableau `want` et de la variable `turn` :

```

1 int turn = 0;
2 bool want[2]={false,false};
3
4 void lock(int i){
5     int other = 1-i;
6     want[i]=true;
7     turn = other;
8     while (want[other] && turn == other) ; //attente active
9 }
10
11 void unlock(int i){
12     want[i]=false;
13 }

```

Il reste à se convaincre que ceci satisfait bien les trois propriétés du mutex :

1. T_0 pose `want[0] = true`
2. T_1 pose `want[1] = true`
3. T_0 pose `turn = 1`
4. T_1 pose `turn = 0`
5. T_0 lit `want[1] = false`

6. T_0 lit `turn`
7. T_1 lit `want[0] = false`
8. T_1 lit `turn`

On sait que

$$\begin{aligned} (1) &< (3) < (5) < (6) \\ (2) &< (4) < (7) < (8) \end{aligned}$$

Exclusion mutuelle Supposons sans perte de généralité que $(3) < (4)$, c.a.d qu'après (4) la valeur de `turn` = 0 et donc à l'étape (8) la condition est fausse.

Supposons par l'absurde que T_0 et T_1 rentrent en section critique alors nécessairement T_1 lit `want[0] = false` car en (8), il lit `turn` = 0. Ainsi $(7) < (1)$. On obtient

$$(1) < (3) < (4) < (7) < (1)$$

Absurde.

Absence de famine Supposons que T_0 soit bloqué en essayant de verrouiller cad. `want[1] = true` et `turn` = 1 indéfiniment. Que se passe-t-il pour T_1 ?

- **1er cas** : il exécute la partie de verrouillage qui précède l'attente, il pose `turn` = 0 et T_0 est débloquent donc impossible.
- **2ème cas** : T_1 est bloqué en attente active, et donc `turn` = 0 (exclu)
- **3ème cas** : T_1 a passé l'attente active, il est entré en section critique et finira par appel `unlock` → poser `want[1]=false` ce qui est exclu.

Absence d'interblocage Absence de famine $\implies$ Absence d'interblocage

Un système de mutex doit nécessairement satisfaire l'exclusion mutuelle, très certainement l'absence d'interblocage et idéalement l'absence de famine.

5.4 Interblocages

Il faudra être très vigilant avec les mutex notamment lorsqu'on en utilise plusieurs car ils peuvent induire des situations d'interblocage où les deux threads se bloquent l'un l'autre comme l'illustre l'exemple suivant :

```

1  let m1 = Mutex.create();;
2  let m2 = Mutex.create();;
3  let f1()=
4      Mutex.lock m1;
5      Mutex.lock m2;
6      Mutex.unlock m2;
7      Mutex.unlock m1;;
8
9  let f2()=
10     Mutex.lock m2;
11     Mutex.lock m1;
12     Mutex.unlock m1;
13     Mutex.unlock m2;;
14
15  let t1 = Thread.create f1 ();;
16  let t2 = Thread.create f2 ();;
17  Thread.join t1;;
18  Thread.join t2;;

```

5.5 Algorithme de la boulangerie de Lamport : synchronisation d'un nombre quelconque de fils d'exécution.

Nous allons maintenant voir un algorithme qui permet d'obtenir l'exclusion mutuelle d'un nombre quelconque de threads fixé. Cet algorithme, tout comme le précédent utilise l'attente active et n'a qu'un intérêt théorique et non pas pratique.

L'idée s'inspire de la gestion de l'attente dans un commerce ou une administration : chaque client (thread) reçoit à son arrivée un ticket avec un numéro de passage et attend son tour (pour entrer en section critique). Lorsqu'il arrive un thread devra obtenir un numéro supérieur à tous les numéros déjà attribués auparavant afin de prévenir l'algorithme de la famine.

Tel quel, l'algorithme ne pourra pas garantir l'exclusion mutuelle car deux threads peuvent acquérir de manière concurrentielle un même numéro et alors décider de rentrer ensemble dans la section critique.

L'algorithme va utiliser deux tableaux de taille n où n est le nombre total de threads. Chaque thread aura un identifiant i et le thread i ne pourra qu'écrire dans les cases i des tableaux. L'un des tableaux, noté `want` contiendra un booléen dans sa case i si le thread i souhaite entrer en section critique. L'autre tableau, noté `label`, contient en case i le numéro de passage obtenu par le thread i .

On peut donc considérer une première version insatisfaisante :

```
1  bool want[n];
2  int label[n];
3
4  void lock(int i){
5      want[i]=true;
6      //phase de recuperation du numero de passage
7      int number = 0;
8      for (int k=0;k<n;k++){
9          if (number<label[k]){
10             number=label[k];
11         }
12     }
13     label[i]=number+1; // phase attente de son tour
14     for (int k =0;k<n;k++){
15         if (k !=i){
16             while(want[k]&&label[k]<label[i]){
17             }
18         }
19     }
20 }
21
22 void unlock(int i){
23     want[i] = false;
24 }
```

Le problème de cette version, comme mentionné précédemment, est que l'exclusion mutuelle n'est pas garantie car deux exécutions concurrentes de la première phase peuvent aboutir à l'obtention d'un même ticket, on se prémunie de ce problème en utilisant l'identifiant des threads pour choisir parmi deux threads ayant le même numéro lequel est autorisé à entrer en section critique avant l'autre.

⇒ Si l'attribution se déroule de manière concurrente, plusieurs threads peuvent obtenir le même label et ainsi décider d'entrer simultanément en même temps en section critique.

```

1  bool want[n];
2  int label[n];
3  void lock(int i){
4      want[i]=true;
5      //phase de recuperation du numero de passage
6      int number = 0;
7      for (int k=0;k<n;k++){
8          if (number<label[k]){
9              number=label[k];
10         }
11     }
12     label[i]=number+1;
13     //phase attente de son tour
14     for (int k =0;k<n;k++){
15         if (k !=i){
16             while(want[k]&&(label[k]<label[i] || (label[k]==label[i]&&k<i))){
17             }
18         }
19     }
20 }
21
22 void unlock(int i){
23     want[i]=false;
24 }

```

Remarque : Cette solution utilise des entiers arbitrairement grand pour des labels, ce qui risque d'être problématique quand on dépasse le plus grand entier machine.

Montrons que ce procédé respecte bien les propriétés attendues :

Exclusion mutuelle Commençons par un petit lemme

Lemme

Pour tout i , la valeur de $label[i]$ est strictement croissante.

DÉMONSTRATION. Lors de la phase d'attribution, la variable $number$ prend en compte la valeur $label[i]$ et donc finira avec une valeur $\geq label[i]$.

Supposons par l'absurde que i et j entrent en section critique simultanément.

T_i	T_j
$want[i] = true$	$want[j] = true$
détermine la valeur finale de $number$	détermine la valeur de $number$
$label[i] = number$	$label[j] = number$
lit $want[j]$	lit $want[i]$
lit $label[j]$	lit $label[i]$

On va noter $labelf[i]$ et $labelf[j]$ les valeurs de $label[i]$ et $label[j]$ après leurs attributions.

Supposons sans perte de généralité :

$$(labelf[i], i) < (labelf[j], j)$$

autrement dit c'est T_i qui devrait rentrer en section critique.

Si T_j entre en section critique alors :

- Soit à l'action " T_j lit `want[i]`" il a lu `want[i] = false` mais dans ce cas T_i n'a pas commencé la phase de détermination (notée (3)) qui doit être précédée par la phase `want[i] = true`. Finalement il obtiendrait en phase (3) une valeur `label[i] = labelf[i] > labelf[j]` car "`label[j] = number`" déjà effectuée quand T_i commence (3).
- Soit à la phase (10), il a lu

$$\text{label}[i] > \text{label}[j] = \text{labelf}[j]$$

avec `label[i] ≠ labelf[i]`.

Ainsi, T_j aurait lu une "ancienne" version de `label[i]` mais d'après le lemme,

$$\text{elle devrait donc être } < \text{labelf}[i] \leq \text{labelf}[j]$$

Interblocage Parmi les threads en train d'essayer de verrouiller, il y en a toujours un qui minimise `(label[i], i)` et qui n'est donc pas bloqué.

Absence de famine Puisque les labels sont attribués de manière strictement croissante, un thread en attente aura un label strictement plus petit qu'un thread qui est passé par la section critique, a déverrouillé et souhaiterait de nouveau verrouiller.

6 Sémaphores

L'exclusion mutuelle vue dans tout ce qui précède garantit qu'un thread seulement peut entrer en section critique.

Le sémaphore est une généralisation du mutex où on garantit qu'un nombre limité de thread peut entrer en section critique.

Le sémaphore est associé à deux fonctions : une fonction d'acquisition qui permet d'accéder à la section critique et une fonction qui permet à un thread de sortir de la section critique.

Fondamentalement, un sémaphore est un compteur (dont la valeur initiale est donnée lors de l'initialisation). Lorsqu'un thread utilise la fonction d'acquisition, si la valeur du compteur est strictement positive alors il se contente de le décrémenter et d'entrer en section critique sinon, si ce dernier est nul, alors il est mis en attente. Quand le thread sort de la section critique, le compteur est incrémenté et s'il y a des threads en attente alors l'un d'eux est réveillé.

6.1 Principe et syntaxe

1. En OCaml

- La fonction `Semaphore.Counting.make : int -> Semaphore.Counting.t` crée un sémaphore dont la capacité est passée en entrée.
- La fonction `Semaphore.Counting.acquire : Semaphore.Counting.t -> unit` attend que le compteur devienne strictement positif puis le décrémente.
- La fonction `Semaphore.Counting.release : Semaphore.Counting.t -> unit` incrémente le compteur et réveille un thread en attente si besoin.

2. En C, on a les fonctions analogues

- La fonction `int sem_init(sem_t *sem, int pshared, unsigned int value)` initialise le sémaphore avec une capacité `v`. L'entier `pshared` spécifie si le sémaphore est ou non partagé : s'il vaut 0, il l'est et ce sera toujours le cas pour nous.
- La fonction `sem_wait(sem_t *s)` met en attente le thread appelant tant que le compteur est nul puis décrémente celui-ci.
- La fonction `sem_post(sem_t *s)` incrémente le compteur et réveille un thread en attente.

6.2 Simulation de Mutex

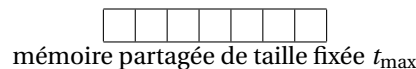
Un sémaphore peut jouer le rôle d'un mutex, pour cela il suffit de choisir la capacité correctement.

```
1 sem_t m;
2 sem_init(&m,0,1);
3 sem_wait(&m);
4 section critique
5 sem_post(&m);
```

6.3 Rendez-vous

```
1 sem_t s;
2
3 void* child(void* arg){
4     printf("childs\n");
5     sem_post(&s);
6     return NULL;
7 }
8
9 int main(int argc, char** argv){
10     sem_init(&s,0,0);
11     printf("parent begins\n");
12     pthread_t c;
13     pthread_create(&c,NULL,child,NULL);
14     sem_wait(&s);
15     printf("parent ends");
16     return 0;
17 }
```

6.4 Producteur, Consommateur



- Les producteurs écrivent et sont mis en attente si la mémoire est pleine.
- Les consommateurs lisent et effacent les éléments et sont mis en attente si la mémoire est vide.

On veut écrire une fonction pour chaque rôle qui seront exécutées chacune par un thread différent chacune par un thread dans le cas d'une file/d'une pile.

Producteur	Consommateur
------------	--------------

Sémaphore plein, vaudra 0 ssi la mémoire est pleine (correspond au nb de cases disponibles).

Sémaphore vide, vaudra 0 ssi la mémoire est vide (correspond au nb de cases occupées) : Initialement : `int fill = 0, int read = 0`.

Version File :

Producteur

```
1 while(true):
2     wait(plein);
3     memoire[fill] = "bla";
4     fill = fill + 1 mod tmax;
5     post(vide);
```

Consommateur

```
1 while(true):
2     wait(vide);
3     memoire[read] = Vide;
4     read = read + 1 mod tmax;
5     post(plein);
```

Invariants

`vide + plein = tmax`
`vide = fill - read`

`vide` = 0 ssi `fill` = `read` ssi la mémoire est vide. Donc il n'y a pas de conflit sur la mémoire car soit `fill` $\neq$ `read`, soit le consommateur est en attente car `vide` = 0.

Version Pile : similaire en ajoutant un mutex pour protéger la mémoire noté `m`.

Producteur

```
1 int c = 0;
2 while (true) :
3     wait(plein);
4     lock(m);
5     memoire[c] = "bla";
6     c = c + 1 ;
7     unlock(m);
8     post(vide);
```

Consommateur

```
1 int c = 0;
2 while (true) :
3     wait(vide);
4     lock(m);
5     memoire[c] = Vide;
6     c = c-1;
7     unlock(m);
8     post(plein);
```

Remarque : Si on intervertissait `lock/wait` ainsi que `unlock/post` on pourrait avoir un interblocage car si par exemple le consommateur verrouille `m` puis attend `vide` qui serait nul, alors le producteur ne peut pas obtenir le verrou et donc ne pourra pas incrémenter `vide` qui restera nul.

6.5 Dîner des philosophes

Soit n philosophes qui mangent sur une table circulaire disposant de n baguettes.

Insérer dessin

L'objectif est de trouver un protocole tel que chaque philosophe correspondra à un thread i permettant à chaque philosophe i de manger avec les baguettes b_i et b_{i-1} (si $i > 0$) et b_0 et b_{n-1} pour le philosophe 0.

On peut écrire une fonction, en fixant pour chaque baguette b_i un mutex m_i .

```
1 philosophe(i) :
2     if (i>0) :
3         lock(m[i-1]);
4         lock(m[i]);
5         MANGE;
6         unlock(m[i]);
7         unlock(m[i-1]);
8     if (i==0) :
9         lock(m[n-1]);
10        lock(m[0]);
11        MANGE;
12        unlock(m[0]);
13        unlock(m[n-1]);
```

Problème : Si tout le monde exécute le verrouillage de la baguette de gauche alors il y a interblocage.

Solution 1

```
1 philosophe(i) :  
2   if (i>0) :  
3     lock(m[i-1]);  
4     lock(m[i]);  
5     MANGE;  
6     unlock(m[i]);  
7     unlock(m[i-1]);  
8   if (i==0) :  
9     lock(m[n-1]);  
10    lock(m[0]);  
11    MANGE;  
12    unlock(m[n-1]);  
13    unlock(m[0]);
```

Ainsi, seules $n - 1$ baguettes sont prises dans le pire des cas au cours de l'opération `lock` et donc il n'y a pas d'interblocage.

Solution 2

On utilise un sémaphore pour garantir que seuls $n - 1$ philosophes qui essaient simultanément d'accéder aux baguettes. On initialise le sémaphore `s` à $n-1$.

```
1 philosophe_non_bloques(i) :  
2   wait(s);  
3   philosophe(i);  
4   post(s);
```

Remarque : Il n'y a rien qui permet de garantir formellement l'absence de famine, il n'est pas exclu qu'un philosophe mange à plusieurs reprises en empêchant ses voisins de manger.

6.6 Coiffeur, barbier

On considère un salon assez petit pour ne posséder qu'une seul siège pour couper les cheveux. Ce salon dispose également d'une salle d'attente avec n sièges.

Coiffeur

Coupe les cheveux

Quand il a terminé, prend quelqu'un dans la salle d'attente
Si la salle d'attente est vide, il s'endort dans son siège

Client

Si on trouve le coiffeur endormi, on le réveille
Sinon, s'il y a de la place, on s'installe dans la salle d'attente
s'il n'y a pas de place, il rentre chez lui

`int libres` qui compte le nombre de sièges libres dans la salle d'attente.

Coiffeur

```
1 while(true) :  
2   if (libres < n) :  
3     libres++ ;  
4     cut();
```

Client

```
1 if (libres > 0) :  
2   libres-- ;  
3   hair_cut();  
4 else:  
5   go_home();
```

1. Protéger `libres` avec un mutex `m`, dans la section **Client** on va insérer `lock(m)` au début et `unlock(m)` dans les deux conditionnelles.
2. Coordination de l'exécution de `hair_cut()` et `cut()`. On va alors créer deux sémaphores : `semaphore BR, CR` initialisés à 0.

Coiffeur

```
1 while(true) :  
2   wait(CR);  
3   lock(m):  
4   libres++ ;  
5   unlock(m);  
6   post(BR);  
7   cut();
```

Client

```
1 lock(m)  
2 if (libres > 0) :  
3   libres-- ;  
4   unlock(m);  
5   post(CR);  
6   wait(BR);  
7   hair_cut();  
8 else:  
9   unlock(m);  
10  go_home();
```

6.7 Barrière de synchronisation

Créer un outil de synchronisation permettant à n threads de s'attendre.

Utilisation type

tâche A en parallèle avec n threads
puis
tâche B en parallèle avec n threads

Une solution insatisfaisante

Créer n threads pour A
faire un jour pour chacun
créer n threads pour B

Cependant, la création de threads est une opération lourde donc on aimerait pouvoir utiliser les mêmes threads pour les 2 tâches en utilisant une barrière.

```
1 type barrier  
2   create_barrier : int -> barrier  
3   wait_barrier : barrier -> unit
```

Une fois ce type créé, on pourra créer n threads qui exécute une fonction de la forme :

partie de tâche a
`wait_barrier(b)`
partie de tâche b

1ère version Barrière avec passage unique : chaque thread n'a le droit qu'à un seul appel à la fonction `wait_barrier`, sinon les conditions souhaitées ne seront pas respectées.

```
1 type barrier = {  
2   size : int ;  
3   mutable waiting : int ;  
4   m : Mutex.t ;  
5   sem_wait : Semaphore.Counting.t  
6 }
```

```

1 let create_barrier (n: int) : barrier =
2   {size = n ; waiting = 0; m = Mutex.create () ; sem_wait = Semaphore.Counting.make 0}
3
4 let wait_barrier (b : barrier) : unit =
5   Mutex.lock b.m ;
6   b.waiting <- b.waiting + 1;
7   if (b.waiting < b.size) then begin
8     Mutex.unlock b.m ;
9     Semaphore.Counting.acquire b.sem_wait
10  end
11  else begin
12    for i = 1 to b.size -1 do
13      Semaphore.Counting.release b.sem_wait
14    done;
15    b.waiting <- 0;
16    Mutex.unlock b.m
17  end

```

Passage unique Par exemple T_0 et T_1 cherchent à utiliser **b**.

T_0	T_1
Incrémente waiting à 1 se met en attente	incrémente waiting à 2 incrémente sem_wait = 1 pose waiting = 0 commence un nouvel appel à wait_barrier pose waiting = 1 passe le acquire car sem_wait = 1

Ainsi, T_1 a franchi 2 fois **b** mais T_0 ne l'a pas passée.

Dans la structure, on rajoute un deuxième sémaphore **gone** initialisé à 0 aussi par **create**.

```

1 let create_barrier (n: int) : barrier =
2   {size = n ; waiting = 0; m = Mutex.create () ; sem_wait = Semaphore.Counting.make 0}
3
4 let wait_barrier (b : barrier) : unit =
5   Mutex.lock b.m ;
6   b.waiting <- b.waiting + 1;
7   if (b.waiting < b.size) then begin
8     Mutex.unlock b.m ;
9     Semaphore.Counting.acquire b.sem_wait ;
10    Semaphore.Counting.release b.gone ;
11  end
12  else begin
13    for i = 1 to b.size -1 do
14      Semaphore.Counting.release b.sem_wait
15    done;
16
17    for i = 1 to b.size - 1 do
18      Semaphore.Counting.acquire gone
19    done;
20    b.waiting <- 0;
21    Mutex.unlock b.m
22  end

```